강화학습 교육 1일차

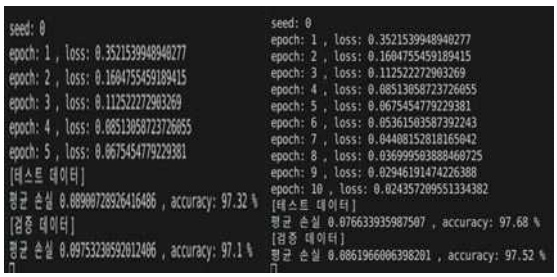
딥러닝과 신경망의 학습 원리

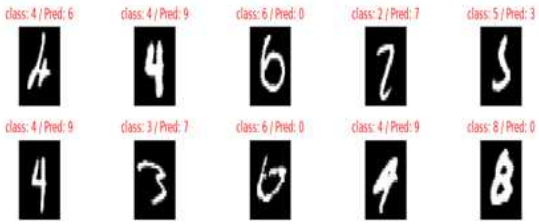
이름 : 김도훈
학번 : 2026406041

<목차>

1. MNIST 손글씨 분류 결과 분석
2. 코드 설명

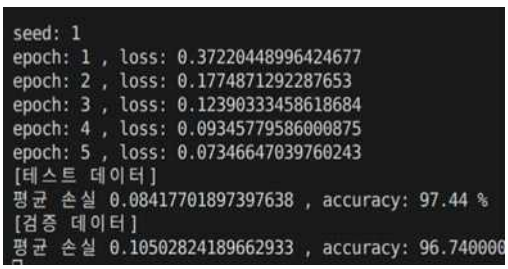
1. MNIST 손글씨 분류 결과 분석

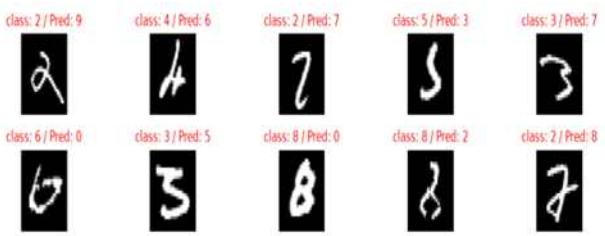




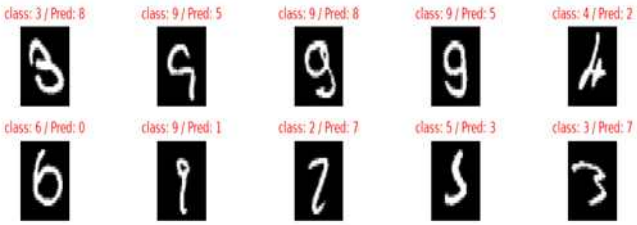
위 두 사진은 epoch을 5로 설정하고 학습한 결과이다. 사진에서는 epoch 횟수가 증가할 때마다 loss가 감소하는 것을 확인할 수 있다. 초기에는 가중치가 적절하지 않아 loss 값이 높다. 하지만 epoch이 증가할 수록 가중치와 편향이 점점 적절한 값으로 수정되기 때문에 loss 값이 감소하는 것이다. epoch을 10으로 했을 때 loss가 점점 감소하는 것을 정확히 확인할 수 있다. 또한 epoch이 10일 때 검증 데이터의 loss가 epoch이 5일 때 보다 작은 것을 확인할 수 있다. 그리고 검증 데이터의 정확도도 epoch이 10일 때 0.42% 높은 것을 확인할 수 있다. 이런 결과 바탕으로 신경망의 성능에는 학습 횟수가 얼마나 큰 영향을 미치는지 알 수 있다. 하지만 학습 횟수를 늘린다고 정확도가 올라가는 것은 보장하지 못한다. 과적합이 발생할 수 있다.

오분류 사진은 임의로 10개를 추출해 분석해 본 결과 오분류가 발생하는 원인은 예측한 숫자의 특징을 오분류 된 데이터가 가지고 있기 때문이다. MNIST 데이터 셋은 다양한 사람들로부터 얻은 것이기 때문에 같은 숫자라도 사람마다 다르다. 오분류 사진의 1행 1열의 사진을 보면 4를 6으로 인식한다. 숫자 6의 특징은 아래에 원이 있고 원 위에 곡선이 우상단으로 뻗는 특징을 가지고 있다. 신경망도 이러한 특징을 학습했다면 1행 1열과 같은 데이터를 숫자 6으로 오분류할 가능성이 존재한다. 다른 오분류 된 데이터들도 거의 동일한 원인에 의해 오분류 되었다고 분석한다.





```
seed: 2
epoch: 1 , loss: 0.36589331008195874
epoch: 2 , loss: 0.16802615140134639
epoch: 3 , loss: 0.11518095864165913
epoch: 4 , loss: 0.08723604261170734
epoch: 5 , loss: 0.06922286313555458
[테스트 데이터]
평균 손실 0.08361121021211147 , accuracy: 97.55 %
[검증 데이터]
평균 손실 0.09694251780509949 , accuracy: 97.14 %
```



모델의 성능을 평가하고 실험을 할 땐 시드를 고정하는 것이 당연한 방법이다. 처음에 코드를 구현할 때 seed를 왜 고정해야 하는지 납득되지 않았다. 실제로 비교해 본 결과 seed가 0, 1, 2일 때 loss와 accuracy 모두 다른 값을 가지는 것을 확인할 수 있다. 실제로 seed는 초기 가중치를 정한다. 초기 상태에는 아무 정보도 없기 때문에 seed를 통해 생성된 난수를 가중치로 사용한다. 그러므로 seed가 달라질 때마다 값이 다른 것이다.

2. 코드 설명(모델 구조, 학습률 선택 이유, 각 루프 설명 포함, GPT 사용한 부분)

```
full = datasets.MNIST("data", train=True, download=True, transform=transforms.ToTensor())
train, val = random_split(full, [55000, 5000], generator=torch.Generator().manual_seed(0))
test = datasets.MNIST("data", train=False, download=True, transform=transforms.ToTensor())
```

위의 코드는 데이터를 불러오는 코드이다. full에 60000개의 데이터를 불러와 저장하고 60000개의 데이터를 train, val에 각각 학습 데이터와 검증 데이터를 55000, 5000개로 나누어 저장한다. 그 이유는 훈련 데이터에 최적화되어 다른 새로운 데이터에서는 예측 성능이 떨어지는 과적합을 방지하기 위해서이다. 그리고 테스트 데이터를 10000개를 test에 저장한다.

```
train_loader = DataLoader(train, batch_size=64, shuffle=True)
val_loader = DataLoader(val, batch_size=256)
test_loader = DataLoader(test, batch_size=256)
```

위의 코드들은 신경망에 배치 사이즈만큼 데이터를 전달하는 코드이다. train_loader는 신경망에 학습 데이터를 전달한다. 학습 데이터 총 55000개 64개씩 묶어 860번 전달한다. val_loa2der는 검증 데이터를 5000개를 256개씩 묶어 보내고 teat_loaer는테스트 데이터 10000개를 256개씩 묶어 신경망에 전달한다.

```
model = nn.Sequential(nn.Flatten(),
                      nn.Linear(784, 128),
                      nn.ReLU(),
                      nn.Linear(128, 10)
                      ).to(device)
```

위의 코드는 신경망 모델을 구성하는 코드이다. 이 모델은 입력층 784개, 은닉층 128개, 출력층 10개로 구성된 MLP 신경망이다. 입력층을 784개로 설정한 이유는 MNIST 데이터의 이미

지 크기가 28 by 28이고 신경망의 입력은 벡터로 입력해야하므로 28 by 28 행렬을 784차원의 벡터로 변환하여 입력하는 것이다. 여기서 변환하기 위해 Flatten() 함수를 사용한다. 그리고 입력층에서는 128개의 데이터를 출력한다. 128은 임의로 정한 숫자이다. Linear(784, 128)에서 입력된 데이터의 특징을 추출하는데 여기서 사용되는 연산이 가중합 연산이다. 입력층은 784개의 데이터를 입력 받고 은닉층은 128개의 데이터를 입력받는다. 이 사이에 서로 다른 가중치를 가진 128개의 뉴런(퍼셉트론)이 존재하고 128번 가중합 연산을 진행하여 은닉층에 128개의 데이터를 입력한다. 그리고 활성화 함수를 거치는데 여기서 사용되는 활성화 함수는 렐루 함수이다. 활성화 함수를 모델에서 사용하는 이유는 비선형성을 추가하기 위해 사용된다. 입력값들이 Linear()만 거친다면 가중합 연산에 의해 나온 결과값이 상수배 되어 선형적으로 표현될 것이다. 그래서 신경망이 표현할 수 있는 값들이 적어지고 예측 성능이 떨어지게 된다. 이를 해결하기 위해 활성화 함수를 사용하는 것이다. 또한 렐루 함수를 사용한 계산 속도가 빠르기 때문이다. 입력값이 양수이면 계산이 필요하지 않고 음수이면 0이기 때문에 다른 활성화 함수들에 비해 계산 속도가 빠른다. 그래서 딥러닝 신경망에 많이 사용된다. 활성화 함수를 통해 128개의 값들을 비선형적으로 변환했다면 숫자 10개를 표현하기 위해 128개의 데이터를 10개로 변환한다. 변환하는 과정은 앞에서 설명한 과정과 동일하다.

```
criterion = nn.CrossEntropyLoss()
```

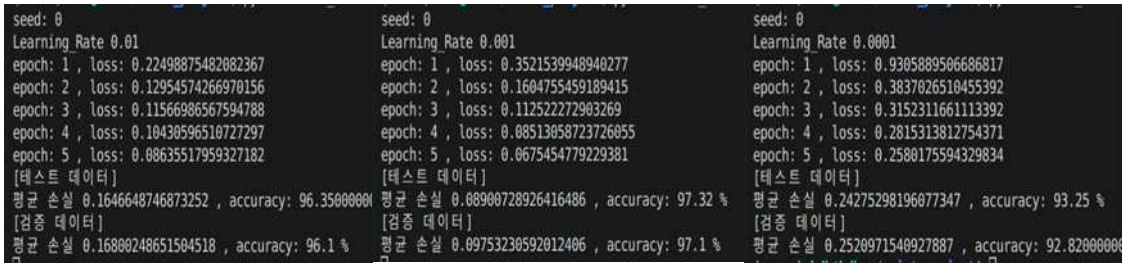
위 코드는 손실 함수를 설정하는 코드이다. 이 코드에서는 분류 문제에 자주 사용되는 CrossEntropy 손실 함수를 사용한다.

$$CrossEntropyloss = \ln(\frac{1}{P}) = \ln 1 - \ln(P) = -\ln(P)$$

정답이 높은 확률일 때 작은 손실을 부여하고 정답이 낮은 확률일 때 큰 손실을 부여하는 크로스 엔트로피 손실 함수에 의해 정답에 높은 확률을 부여하도록 모델이 학습된다. 역전파 과정에서 손실을 줄이도록 가중치를 수정하기 때문에 정답이 아닌 것에 낮은 확률을 부여해 손실을 크게 만드는 것이다. 이런 이유로 분류 문제에서 자중 사용된다.

```
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
```

위 코드는 역전파 시 사용할 최적화 알고리즘과 학습률을 설정하는 코드이다. 이 코드에서는 최적화 알고리즘을 Adam을 사용하고 학습률은 1e-3 = 1 * 10^-3 = 0.001을 사용한다. 학습률로 0.001을 사용하는 이유는 너무 작은 학습률은 손실 함수의 최소값에 너무 느리게 수렴하고 너무 큰 학습률은 손실 함수의 최소값을 찾지 못하고 발산해 버릴 수 있기 때문이다. 실제 실험 결과로



learning rate가 0.001일 때 가장 높은 정확도를 보이고 있다.

```
#학습 횟수
epochs = 5
for epoch in range(epochs):
    model.train() #학습 모드
    total_loss = 0.0 #반복문이 돌때마다 초기화
    for images, labels in train_loader:
        # 설정한 연산 장치로 이동(이 코드에서는 CPU)
        images = images.to(device)
        labels = labels.to(device)
        optimizer.zero_grad() #기울기 초기화
        logits = model(images) #순전파 28 * 28 -> 784 -> 128 -> 10
        loss = criterion(logits, labels) #위에서 설정한 손실 함수로 손실값 계산
        # 역전파
        loss.backward() # 기울기 계산
        optimizer.step() # 가중치와 편향 수정
        total_loss += loss.item() * images.size(0)
        train_loss = total_loss / len(train)
```

위 코드는 실제 신경망을 학습시키는 루프이다. 이 루프는 epoch 만큼 반복한다. 즉 epoch 이 5라면 55000개의 학습 데이터를 5번 학습하는 것이다. 모델을 학습 모드로 전환시킨 후 각 반복마다 누적 손실과 기울기를 초기화한다. 그리고 784개의 데이터가 입력층 -> 은닉층 -> 출력층을 거쳐 10개의 logits을 갖는다. 이 logits과 label을 차를 통해 손실을 계산한다. 이 과정이 순전파이다. 그리고 손실을 줄이기 위해 출력층에서 입력층까지 연쇄 법칙을 사용해 역전파를 진행한다. optimizer.step()을 사용해 가중치를 업데이트 한다. 이 과정이 역전파이다. 이 과정을 epoch 만큼 반복한다.

```
# 성능 평가 함수
def evaluate(loader):
    model.eval() #평가 모드
    loss_sum, correct, count = 0.0, 0, 0
    with torch.no_grad():
        for images, labels in loader:
            images, labels = images.to(device), labels.to(device)
            logits = model(images)
            loss_sum += criterion(logits, labels).item() * len(labels)
            correct += (logits.argmax(1) == labels).sum().item()
            count += len(labels)
    return loss_sum / count, correct / count
```

위 코드는 성능 평가 함수이다. 이 코드는 실제 테스트를 해보는 함수이다. 이 함수에서는 학습된 모델을 평가 모드로 사용해 손실을 계속 누적하여 전체 손실, 모델이 정답을 맞힌 이미지의 개수, 평가한 전체 이미지 개수를 계산한고 retrun 값을 받아 출력한다. 이 루프는 불어온 데이터 / 배치 사이즈 만큼 반복한다.

```
# 오분류 사례 분석
model.eval()
wrong_images = [] # 잘못 분류된 이미지
wrong_labels = [] # 실제 정답 숫자
wrong_preds = [] # 모델이 예측한 숫자
with torch.no_grad():
    for images, labels in test_loader:
        #각각 이미지와 정답 256개씩 연산 장치로 이동
        images = images.to(device) #이미지
        labels = labels.to(device) #정답
        # 모델 예측
        logits = model(images)
        preds = logits.argmax(dim=1) # 10개의 로짓중 최댓값을 찾음
        # 오분류된 이미지 찾기
        wrong_mask = preds != labels #정답과 예측값이 다르면 trun, 같으면 false
        # 오분류 결과 저장
        # trun 인 것들만 저장
        wrong_images.extend(images[wrong_mask].cpu())
        wrong_labels.extend(labels[wrong_mask].cpu().tolist())
        wrong_preds.extend(preds[wrong_mask].cpu().tolist())
        # 10개 이상 찾으면 종료
        if len(wrong_images) >= 10:
            break
```

오분류 사례를 찾는 코드와 오분류 이미지를 시각화하는 코드를 구현할 때 GPT의 도움을 받았다. 파이썬을 처음 접해보기도하고 코드를 구현할 때 오분류한 것을 어떻게 판단할지 로직을 구상하고 코드까지 구현하는 것에 한계가 있었다. 그래서 GPT에게 정보를 제공 받아 코드와 로직을 이해하는 것이 나에서 더 도움이 될 거 같아 GPT에게 도움을 받았다.
이 코드는 오분류된 이미지를 찾는 코드이다. 다른 코드와 크게 다를 것 없이 모델을 평가 모드로 불러오고 예측을 진행한다. 그리고 예측한 숫자와 정답을 비교하여 맞다면 false, 틀리면 true를 잘못 분류된 이미지의 배열에 저장한다. 그리고 true 인것들만 이미지, 라벨, 예측 값을 저장한다. 이 과정을 통해 잘못 분류된 이미지를 10개 이상 찾았을 때 루프를 종료한다.

```
# 오분류 이미지 10개 출력, 2행 5열 10개
fig, axes = plt.subplots(2, 5, figsize=(12, 5))
for i, ax in enumerate(axes.flat):
    if i >= min(10, len(wrong_images)):
        ax.axis("off")
        continue
    ax.imshow(wrong_images[i].squeeze(), cmap="gray")
```

```
        ax.set_title(f"class: {wrong_labels[i]} / Pred: {wrong_preds[i]}", color="red")
        ax.axis("off")
plt.tight_layout()
plt.show()
```

그리고 위의 코드를 사용해 오분류된 이미지, 라벨, 예측값을 2행 5열 형태로 출력한다.